Instead, exception filters let you handle the exception in the filter pipeline and generate an appropriate response body for API clients. The exception handler middleware intercepts only errors without a body, so it will let the modified web API response pass untouched.

NOTE The `[ApiController]` attribute converts error `StatusCodeResult`s to a `ProblemDetails` object, but it doesn't catch exceptions.

Exception filters can catch exceptions from more than your action methods and page handlers. They'll run if an exception occurs at these times:

- During model binding or validation
- When the action method or page handler is executing
- When an action filter or page filter is executing

You should note that exception filters won't catch exceptions thrown in any filters other than action and page filters, so it's important that your resource and result filters don't throw exceptions. Similarly, they won't catch exceptions thrown when executing an `IActionResult`, such as when rendering a Razor view to HTML.

Now that you know why you might want an exception filter, go ahead and implement one for `RecipeApiController`, as shown next. This lets you safely remove the `try-catch` block from your action methods, knowing that your filter will catch any errors.

Listing 22.8 The `HandleExceptionAttribute` exception filter

```
public class HandleExceptionAttribute : ExceptionFilterAttribute          ❶
{
    public override void OnException(ExceptionContext context)           ❷
    {
```